Subrat Das — Premium Portfolio + Admin CMS

A world-class, recruiter-ready personal portfolio for **Subrat Das** (MCA student, KIIT University, Bhubaneswar) with a complete **Admin CMS** that lets you manage every piece of content from a dashboard — no code edits required.

> Built with React 19, Vite, TypeScript, Tailwind CSS v4, Framer Motion, React Router, React Hook Form, Zod, and a Supabase-compatible data layer.

✨ Features

Public Portfolio

- **Animated Hero** with a live canvas particle-network background, typing effect, availability status, floating stat cards, and social icons.
- **About, Skills** (animated, category-filtered cards), **Education** & **Experience** timelines.
- **Projects** with rich detail modals: screenshots, architecture diagrams, features, challenges, learnings, technologies, GitHub & live links, view counts.
- **Certifications** with verification links, **Achievements**, **Coding Profiles** (LeetCode, GitHub, etc.).
- **Live GitHub Integration** — auto-fetches pinned repos, followers, stars, language distribution, and recent commits from the GitHub REST API.
- **Blog**, **Testimonials**, **Gallery** (filterable, fullscreen lightbox).
- **Resume** download/view with view + download counters and DD/MM/YYYY upload date.
- **Contact form** that stores messages in the database and emails the owner.
- **AI Voice Assistant** — floating chat with speech recognition (input) + text-to-speech (output), answers questions about Subrat using live portfolio data, navigates sections, opens GitHub, downloads resume.
- Fully **responsive**, **accessible** (WCAG-minded: reduced-motion, semantic markup, keyboard-friendly), **SEO-optimized** meta tags, and **lazy-loaded** sections for performance.

Admin CMS (`/admin`)

Secure email/password authentication, protected routes, session management, logout.

Modules (full CRUD where applicable):

- Dashboard overview with live analytics & quick actions
- Profile, Hero, About, Skills, Education, Experience, Projects, Certificates, Achievements, Coding Profiles, Gallery, Testimonials, Blogs, Social Links, Resume
- **Dynamic Page Builder** — drag-to-reorder sections, show/hide, add custom sections (Publications, Patents, Talks…), **Save Draft**, **Publish**, **Preview**.
- **Messages** — read, reply (email + saved in-app), mark read/unread, star, delete, **export CSV**, Gmail/IMAP sync.
- **Visitor Intelligence** — every page view, project view, resume view/download is enriched server-side with IP, city/region/country, ISP, device and browser; click a row to highlight it (recruiter follow-up) — the highlight persists across refreshes and devices until you turn it off; filter to “Highlighted only”, delete single or selected rows.
- **Analytics** — page views, project views, resume downloads, contact requests, assistant interactions, traffic chart, top referrers/sources, per-project view counts.
- **Audit Logs** — every create/update/delete/login is tracked.
- **Settings** — SEO title/description/keywords, theme colors, fonts, OG image, analytics toggle.

🔐 Admin Credentials (demo)

```
Email:    admin@subrat.dev
Password: admin123
```

Visit `/admin/login`.

🏗️ Architecture

```
My_Portfolio/
├── api/                  # Vercel serverless functions
│   ├── track.ts          # Enriched visitor tracking (IP, geo, ISP, device → analytics.meta)
│   ├── send-email.ts     # Send admin replies via Gmail SMTP
│   ├── sync-email.ts     # IMAP sync — pulls email replies into the CMS
│   └── keepalive.ts      # Supabase keepalive ping
├── docs/                 # Documentation (README.pdf, workflow & architecture PDFs)
├── public/               # Static assets (images, logos, certs, projects)
├── supabase/             # SQL schema + incremental migrations
│   ├── schema.sql        # Full schema: tables, RLS, storage, realtime
│   ├── add-analytics-delete.sql
│   ├── add-analytics-highlight.sql
│   ├── add-contact-replies.sql
│   ├── add-reset-functions.sql
│   ├── add-resume-views.sql
│   ├── update-skill-categories.sql
│   └── fix-resume-downloads.sql
├── src/
│   ├── lib/
│   │   ├── types.ts          # Domain types (mirror the SQL schema)
│   │   ├── store.ts          # Reactive data layer (Supabase + localStorage + offline queue)
│   │   ├── supabase.ts       # Supabase client + auth helpers
│   │   ├── seed.ts           # Initial content for Subrat Das
│   │   ├── github.ts         # GitHub REST API integration
│   │   ├── assistant.ts      # AI assistant logic + Web Speech API
│   │   ├── fieldConfigs.ts   # Field schemas driving the generic CMS editors
│   │   ├── visitor.ts        # Visitor meta parsing (IP, geo, ISP, device formatting)
│   │   ├── highlights.ts     # Persistent analytics highlights (localStorage + cloud sync)
│   │   ├── imageHost.ts      # Image compression + Supabase Storage upload
│   │   └── emojis.ts         # Emoji picker categories
│   ├── hooks/
│   │   └── useStore.ts       # Reactive store hooks (useSyncExternalStore)
│   ├── components/
│   │   ├── ui/               # Reusable primitives (Reveal, Modal, Skeleton, ParticleField…)
│   │   ├── public/           # Portfolio sections + SectionRenderer (dynamic page builder)
│   │   └── admin/            # AdminLayout, CollectionEditor, SingletonEditor, fields
│   └── pages/
│       ├── PublicSite.tsx
│       ├── AdminApp.tsx
│       └── admin/            # Dashboard, PageBuilder, Messages, Analytics, AuditLogs, Login, ResetCallback
└── README.md / LICENSE / vercel.json / vite.config.ts / package.json
```

Tracking & analytics flow

```
Visitor → trackEnrichedEvent()  (src/lib/store.ts)
        ├─ optimistic counters (projects.views / resume.views / resume.downloads)
        ├─ Supabase RPC increment (incr_project_views / incr_resume_views / incr_resume_downloads)
        └─ POST /api/track → ipwho.is + Vercel geo headers
```

```
                          → analytics row { type, path, referrer, meta: JSON{ip,country,region,city,isp,device} }
        → Admin Dashboard / Analytics (live, delete, highlight, bulk-delete)
```

Data layer → Supabase

`src/lib/store.ts` mirrors the Supabase client API (`from(table).select/insert/update/delete`) and keeps a local cache for instant UI updates:

1. Optimistic local write → `emit()` for immediate UI refresh
2. `pushTable()` queues the cloud write (insert/update/delete/upsert/reorder)
3. Failed writes are retried by `retryFailedOps()` ; pending deletes are never resurrected by a later sync
4. `syncFromCloud()` merges cloud rows with local pending ops, then persists to localStorage

Run the SQL files in `supabase/` in your Supabase project (schema + migrations for analytics delete, contact replies, reset RPCs, skill categories, resume views).

The rest of the app needs **no changes**.

🗄️ Database

`supabase/schema.sql` contains the full PostgreSQL schema with:

- 20 tables (profile, hero, about, skills, education, experience, projects, certificates, achievements, gallery, testimonials, blogs, coding_profiles, social_links, contact_messages, contact_replies, resume, settings, sections, analytics, audit_logs)
- Foreign keys, indexes
- **Row Level Security** policies (public read, admin write, public analytics insert, admin analytics delete/highlight)
- Storage bucket + policies for resume/images
- `updated_at` trigger
- Counter RPCs: `incr_project_views` , `incr_resume_views` , `incr_resume_downloads`
- Admin RPCs: `reset_*` , `reset_all_counters` , `delete_analytics` , `set_analytics_highlight` , `add_contact_reply`

Migrations (run once in Supabase SQL Editor)

File	Purpose
`add-contact-replies.sql`	Reply thread table + RLS
`add-reply-insert.sql`	Guaranteed reply insert / `add_contact_reply` RPC
`add-resume-views.sql`	Resume view counter
`add-counter-functions.sql`	View/download increment RPCs
`add-reset-functions.sql`	Admin reset RPCs
`add-analytics-delete.sql`	Admin delete policy + `delete_analytics` RPC
`add-analytics-highlight.sql`	`highlighted` column + update policy + `set_analytics_highlight` RPC (persistent highlights)
`update-skill-categories.sql`	Framework / Other skill categories (FastAPI, LLM Basics)
`fix-resume-downloads.sql`	Correct `incr_resume_downloads()` (adds missing WHERE)

File	Purpose
`enable-realtime.sql`	Realtime publication for live admin updates
`add-sort-order.sql`	`sort_order` column for drag ordering

🚀 Getting Started

```
npm install
npm run dev      # start dev server
npm run build    # production build
npm run preview  # preview the build
```

🛠️ Tech Stack

Layer	Tech
Frontend	React 19, Vite, TypeScript, Tailwind CSS v4, Framer Motion, React Router, React Hook Form, Zod, Lucide React
Backend	Supabase (PostgreSQL, Auth, Storage, RLS) — schema + migrations in `supabase/`
Serverless	Vercel Functions (`/api/track` , `/api/send-email` , `/api/sync-email` , `/api/keepalive`)
Email	Nodemailer (Gmail SMTP) for replies, IMAP for syncing sent mail
Integrations	GitHub REST API, Web Speech API (voice assistant), ipwho.is (IP geolocation)

🌐 Environment Variables

Client (`.env`):

```
VITE_SUPABASE_URL=...
VITE_SUPABASE_ANON_KEY=...
```

Server (Vercel → Project Settings → Environment Variables):

```
SUPABASE_URL, SUPABASE_ANON_KEY
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS     # sending replies
IMAP_HOST, IMAP_PORT, IMAP_USER, IMAP_PASS     # syncing email replies
OWNER_EMAIL                                    # contact-form notifications
```

📁 Documentation

- Full README (PDF)

- Project Structure, Workflow & Architecture (PDF)
- Project Structure, Workflow & Architecture (Markdown)

📝 Notes

- The hero uses a GPU-friendly canvas particle network (reduced-motion aware) rather than a heavy Three.js bundle for reliability and performance.
- All content is seeded with realistic data for Subrat Das and is fully editable from the CMS.
- GitHub data is cached for 30 minutes client-side to respect rate limits.

📄 License

This project is licensed under the MIT License - see the LICENSE file for details.

© 2026 Subrat Das. All rights reserved. This is a personal portfolio project - please do not copy or redistribute without permission.